

PAST_EXAMS.md

CCF CSP 近年真题汇编

> ****说明****：本文档收录 2020 – 2023 年 CCF CSP 认证考试中的经典题目，提供完整题目描述、样例、解题思路与正确 C++17 代码，并标注算法类型，与[算法速查手册](ALGORITHMS.md)联动。
> 题目内容根据公开资料整理，以 CCF 官网和各大 OJ 平台（洛谷等）的原题为准。
> ****2024 – 2025 年真题****：CCF 官方网站（cspro.org）可以下载历年题目和测试数据。

题目索引

届次	题号	题目名称	难度	算法类型	算法手册链接
2023年9月（第31次）	T1	[坐标变换(I)](#202309-1坐标变换i)		数学 / 模拟 / 旋转缩放	[§ 1 暴力枚举/模拟](ALGORITHMS.md#1-暴力枚举--模拟)
2023年3月（第30次）	T2	[垦田计划](#202303-2垦田计划)		二分答案	[§ 2 二分查找/二分答案](ALGORITHMS.md#2-二分查找--二分答案)
2022年9月（第28次）	T1	[如此编码](#202209-1如此编码)		模拟 / 混合进制	[§ 1 暴力枚举/模拟](ALGORITHMS.md#1-暴力枚举--模拟)
2022年3月（第27次）	T1	[归一化处理](#202203-1归一化处理)		数学 / 模拟	[§ 1 暴力枚举/模拟](ALGORITHMS.md#1-暴力枚举--模拟)
2022年3月（第27次）	T2	[训练计划](#202203-2训练计划)		拓扑排序 / 关键路径	[§ 7 图论](ALGORITHMS.md#7-图论基础)
2021年9月（第26次）	T1	[数组推导](#202109-1数组推导)		贪心 / 前缀最大值	[§ 3 贪心算法](ALGORITHMS.md#3-贪心算法)
2021年9月（第26次）	T2	[非零段划分](#202109-2非零段划分)		差分数组 / 枚举	[§ 8 前缀和/差分](ALGORITHMS.md#8-前缀和--差分)
2021年3月（第25次）	T1	[序列查询](#202103-1序列查询)		数学 / 模拟	[§ 1 暴力枚举/模拟](ALGORITHMS.md#1-暴力枚举--模拟)
2020年12月（第24次）	T2	[期末预测之最佳阈值](#202012-2期末预测之最佳阈值)		排序 / 前缀和 / 枚举	[§ 8 前缀和/差分](ALGORITHMS.md#8-前缀和--差分)

> 难度说明： = T1 级（前100分）， = T2 级（100分）， = T3 级（进阶）

202209-1：如此编码

****届次****：2022年9月（第28次 CSP-J/S）第一题
****满分****：100 分
****算法类型****： ****模拟 / 混合进制解码****
****算法手册****：→ [§ 1 暴力枚举 / 模拟](ALGORITHMS.md#1-暴力枚举--模拟)

题目描述

给定正整数序列 $b_1, b_2, \ldots, b_n$ （每个 $b_i \geq 2$ ），定义混合基数编码：

$$B_0 = 1, \quad B_i = B_{i-1} \times b_i$$

一个满足 $0 \leq a_i < b_i$ 的 n 元组 $(a_1, a_2, \dots, a_n)$ 编码为：

$$x = a_1 \cdot B_0 + a_2 \cdot B_1 + \dots + a_n \cdot B_{n-1}$$

现给定编码后的整数 m （即 x ）和基数序列 b ，请还原原始数组 a 。

输入格式

第一行：两个正整数 n （ $1 \leq n \leq 20$ ）和 m （ $0 \leq m < B_n$ ）

第二行： n 个正整数 $b_1, b_2, \dots, b_n$ （ $2 \leq b_i \leq 100$ ）

输出格式

一行，输出 n 个整数 $a_1, a_2, \dots, a_n$ ，空格分隔。

样例

****输入****

'''

3 23

2 5 4

'''

****输出****

'''

1 1 2

'''

****验证****： $a_1 \cdot 1 + a_2 \cdot 2 + a_3 \cdot 2 \times 5 = 1 \times 1 + 1 \times 2 + 2 \times 10 = 23 \quad \checkmark$

解题思路

这是经典的****混合进制解码****，类比十进制中的模运算：

- $a_1 = m \bmod b_1$ ，然后 $m \leftarrow m / b_1$

- $a_2 = m \bmod b_2$ ，然后 $m \leftarrow m / b_2$

- ……以此类推

每一步取余得到当前位的值，整除消去已处理的部分。

****关键点****： m 可能很大（最大约 100^{20} ），需要使用 `long long`（ $\leq 2^{63}-1 \approx 9.2 \times 10^{18}$ ）。当 $n=20$ ，最坏情况约 100^{20} 远超 `long long`，但题目保证 $m < B_n$ ，实际约束在合理范围内（一般 $n \leq 15$ ）。

完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 long long m;
 cin >> n >> m;

 vector<long long> b(n);
 for (auto& x : b) cin >> x;

 for (int i = 0; i < n; i++) {
 cout << m % b[i];
 m /= b[i];
 if (i < n - 1) cout << ' ';
 }
 cout << '\n';
 return 0;
}
```
```

****复杂度****： $O(n)$ 时间， $O(n)$ 空间。

202203-1：归一化处理

****届次****：2022年3月（第27次 CSP-J/S）第一题

****满分****：100 分

****算法类型****： ****数学 / 模拟****

****算法手册****：→ [§ 1 暴力枚举 / 模拟](ALGORITHMS.md#1-暴力枚举--模拟)

题目描述

给定 n 个浮点数 $x_1, x_2, \ldots, x_n$ ，对其进行****标准化（Z-score 归一化）****处理：

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

$$y_i = \frac{x_i - \mu}{\sigma}$$

输出归一化后的数列 $y_1, y_2, \ldots, y_n$ ，每个数保留 6 位小数。

题目保证所有 x_i 不完全相同（即 $\sigma > 0$ ）。

输入格式

第一行：正整数 n ($2 \leq n \leq 100$)

接下来 n 行，每行一个浮点数 x_i ($-10^4 \leq x_i \leq 10^4$ ，精度到小数点后 4 位)

输出格式

n 行，第 i 行输出 y_i ，保留 6 位小数。

样例

****输入****

```
4
-1.0
-0.5
0.5
1.0
```

****输出****

```
-1.341641
-0.447214
0.447214
1.341641
```

****验证****： $\mu = 0$ ， $\sigma = \sqrt{(1+0.25+0.25+1)/4} = \sqrt{0.625} \approx 0.7906$ ， $y_1 = (-1-0)/0.7906 \approx -1.341641$ ✓

解题思路

直接按公式模拟：

1. 一次遍历求均值 μ
2. 二次遍历求方差，取平方根得 σ
3. 三次遍历输出 $y_i = (x_i - \mu) / \sigma$

注意使用 `double` 精度，输出时用 `fixed << setprecision(6)`。

完整代码

```
```cpp
```

```

#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 cin >> n;
 vector<double> x(n);
 for (auto& v : x) cin >> v;

 // 计算均值
 double mu = 0;
 for (double v : x) mu += v;
 mu /= n;

 // 计算总体标准差
 double var = 0;
 for (double v : x) var += (v - mu) * (v - mu);
 double sigma = sqrt(var / n);

 // 输出归一化结果
 cout << fixed << setprecision(6);
 for (int i = 0; i < n; i++) {
 cout << (x[i] - mu) / sigma << "\n";
 }
 return 0;
}

```

**\*\*复杂度\*\***： $O(n)$  时间， $O(n)$  空间。

---

## 202203-2：训练计划

**\*\*届次\*\***：2022年3月（第27次 CSP-J/S）第二题

**\*\*满分\*\***：100 分

**\*\*算法类型\*\***：           **\*\*拓扑排序 / 逆向关键路径\*\***

**\*\*算法手册\*\***：→ [ § 7 图论（拓扑排序） ](ALGORITHMS.md#7-图论)

---

### 题目描述

共有  $n$  个训练项目（编号  $1 \sim n$ ），每个项目需要  $t_i$  天完成。

部分项目有**\*\*前置项目\*\***  $\text{pre}_i$ ：必须在项目  $\text{pre}_i$  完成后才能开始项目  $i$ （每个项目最多一个前置项目，构成若干棵有根树）。

总训练周期为  $d$  天，所有项目必须在第  $d$  天结束（含第  $d$  天）。

请计算每个项目\*\*最晚\*\*可以从第几天开始，使得所有项目仍能在第  $d$  天前完成。

### ### 输入格式

第一行：两个正整数  $n$  ( $1 \leq n \leq 300$ ) 和  $d$  ( $1 \leq d \leq 300$ )

接下来  $n$  行，第  $i$  行两个整数  $t_i$  ( $1 \leq t_i \leq d$ ) 和  $\text{pre}_i$  ( $0$  表示无前置项目)

### ### 输出格式

$n$  行，第  $i$  行输出项目  $i$  最晚开始天数。

### ### 样例

**\*\*输入\*\***

```
5 7
3 0
2 1
4 1
1 2
2 0
```

**\*\*输出\*\***

```
1
5
4
7
6
```

**\*\*说明\*\*：**

- 项目1：最晚第1天开始，第3天结束
- 项目2：最晚第5天开始（1完成后=第4天起），第6天结束
- 项目3：最晚第4天开始（1完成后=第4天起），第7天结束
- 项目4：最晚第7天开始（2完成后=第7天起），第7天结束
- 项目5：无前置，最晚第6天开始，第7天结束

---

### ### 解题思路

**\*\*逆向 BFS + 关键路径\*\*。**

**\*\*正向思维\*\*：**最早开始时间（前向拓扑排序），保证满足所有前置条件。

**\*\*逆向思维\*\*：**最晚开始时间（反向拓扑排序），从叶节点往根方向推算。

对于每个项目  $i$ ，设  $\text{latest}[i]$  为最晚开始天数：

- \*\*叶节点\*\*（无后继）： $\text{latest}[i] = d - t_i + 1$
- \*\*非叶节点\*\*（有后继  $j_1, j_2, \dots$ ）：项目  $i$  必须在最早的后继开始前完成
$$\text{latest}[i] = \min_{j \in \text{succ}(i)} \text{latest}[j] - t_i$$

实现：统计每个节点的出度，出度为 0 的节点先入队，按逆拓扑顺序处理。

---

### 完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d;
    cin >> n >> d;

    vector<int> t(n + 1), pre(n + 1, 0);
    vector<int> outdeg(n + 1, 0); // 每个节点的后继数（出度）

    for (int i = 1; i <= n; i++) {
        cin >> t[i] >> pre[i];
        if (pre[i]) outdeg[pre[i]]++; // pre[i] 多了一个后继
    }

    vector<int> latest(n + 1, INT_MAX); // 初始化为"无穷大"
    queue<int> q;

    // 叶节点（无后继）：最晚开始时间 = d - t[i] + 1
    for (int i = 1; i <= n; i++) {
        if (outdeg[i] == 0) {
            latest[i] = d - t[i] + 1;
            q.push(i);
        }
    }

    // 逆向 BFS：从叶节点向根推算
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        int p = pre[u];
        if (p == 0) continue; // u 是根节点
        // u 的前置项目 p：p 必须在 latest[u] 前完成，即 latest[p] + t[p] - 1 < latest[u]
        latest[p] = min(latest[p], latest[u] - t[p]);
    }
}
```

```

    if (--outdeg[p] == 0) q.push(p); // p 的所有后继都处理完, p 可以入队
}

for (int i = 1; i <= n; i++) {
    cout << latest[i] << '\n';
}
return 0;
}
'''

```

****复杂度****： $O(n)$ 时间， $O(n)$ 空间。

202109-1：数组推导

****届次****：2021年9月（第26次 CSP-J/S）第一题

****满分****：100 分

****算法类型****： ****贪心 / 前缀最大值分析****

****算法手册****：→ [§ 3 贪心算法](ALGORITHMS.md#3-贪心算法)

题目描述

给定一个非严格递增序列 $B_1 \leq B_2 \leq \dots \leq B_n$ ，其中 B_i 是数组 a 的****前缀最大值****：

$$B_i = \max(a_1, a_2, \dots, a_i)$$

已知每个 a_i 满足 $0 \leq a_i \leq B_i$ 。

请计算数组 a 的****元素和****的最大值与最小值。

输入格式

第一行：正整数 n ($1 \leq n \leq 1000$)

第二行： n 个非负整数 $B_1, B_2, \dots, B_n$ ($0 \leq B_i \leq 10^9$ ，非严格递增)

输出格式

两行：第一行输出最大和，第二行输出最小和。

样例

****输入****

'''

5

3 3 4 4 5

'''

****输出****

```

19

12

```

****说明**：**

- ****最大化****：令 $a = [3, 3, 4, 4, 5]$ ，每个 $a_i = B_i$ ，和 $= 3+3+4+4+5 = 19$

- ****最小化****：

- $a_1 = B_1 = 3$ （必须等于 B_1 ）

- $a_2 = 0$ （ $B_2 = B_1 = 3$ ，无需 a_2 贡献新最大值）

- $a_3 = B_3 = 4$ （ $B_3 > B_2$ ，必须由 a_3 贡献新最大值）

- $a_4 = 0$ （ $B_4 = B_3 = 4$ ）

- $a_5 = B_5 = 5$ （ $B_5 > B_4$ ，必须贡献新最大值）

- 最小和 $= 3+0+4+0+5 = 12$

解题思路

****最大化****：令所有 $a_i = B_i$ ，这样 $\max(a_1, \dots, a_i) = B_i$ 仍然成立。

$$\text{maxSum} = \sum_{i=1}^n B_i$$

****最小化****：分析每个 a_i 最小能取多少：

- a_1 必须等于 B_1 （ $\max(a_1) = B_1$ 必须成立）

- 对于 $i > 1$ ：

- 若 $B_i > B_{i-1}$ ： a_i 必须等于 B_i （只能由 a_i 贡献新的前缀最大值）

- 若 $B_i = B_{i-1}$ ： a_i 可以取 0 （前缀最大值已由前面的元素保证）

$$\text{minSum} = B_1 + \sum_{i=2}^n [B_i > B_{i-1}] \cdot B_i$$

完整代码

```cpp

#include <bits/stdc++.h>

using namespace std;

int main() {

ios::sync\_with\_stdio(false);

cin.tie(nullptr);

int n;

cin >> n;

vector<long long> B(n);

for (auto& v : B) cin >> v;

```

long long maxSum = 0, minSum = B[0]; // a[0] 必须等于 B[0]
for (int i = 0; i < n; i++) {
 maxSum += B[i];
 if (i > 0 && B[i] > B[i - 1]) {
 minSum += B[i]; // a[i] 必须等于 B[i]
 }
 // 若 B[i] == B[i-1], a[i] 取 0, 不加
}

cout << maxSum << '\n' << minSum << '\n';
return 0;
}
'''

```

**\*\*复杂度\*\***： $O(n)$  时间， $O(n)$  空间。

---

## 202109-2：非零段划分

**\*\*届次\*\***：2021年9月（第26次 CSP-J/S）第二题

**\*\*满分\*\***：100 分

**\*\*算法类型\*\***：      **\*\*差分数组 / 枚举\*\***

**\*\*算法手册\*\***：→ [ § 8 前缀和 / 差分 / 线段树](ALGORITHMS.md#8-前缀和--差分--线段树)

---

### 题目描述

给定  $n$  个正整数  $a_1, a_2, \dots, a_n$  ( $1 \leq a_i \leq n$ )。

选取正整数阈值  $p$  ( $1 \leq p \leq n+1$ )，将所有\*\*小于\*\*  $p$  的元素替换为  $0$ ，得到新序列。

定义\*\*非零段\*\*为新序列中极大的连续正整数子序列（每段所有元素均  $> 0$ ）。

求使非零段数量最多的最小阈值  $p$ 。

### 输入格式

第一行：正整数  $n$  ( $1 \leq n \leq 5 \times 10^4$ )

第二行： $n$  个正整数  $a_1, a_2, \dots, a_n$

### 输出格式

一行，输出最优阈值  $p$ 。

### 样例

**\*\*输入\*\***

'''

```
6
4 2 1 3 2 4
'''
```

```
输出
'''
```

```
3
'''
```

```
说明：
```

- $p=1$ ：序列  $[4,2,1,3,2,4]$ ，非零段 = 1
- $p=2$ ：序列  $[4,2,0,3,2,4]$ ，非零段 = 2 ( $[4,2]$  和  $[3,2,4]$ )
- $p=3$ ：序列  $[4,0,0,3,0,4]$ ，非零段 = 3 ( $[4]$ 、 $[3]$ 、 $[4]$ ) ✓ 最多
- $p=4$ ：序列  $[4,0,0,0,0,4]$ ，非零段 = 2

```

```

### ### 解题思路

**核心观察**：位置  $i$  是某个非零段的**左端点**，当且仅当：  
 $a_i \geq p \wedge (i=1 \vee a_{i-1} < p)$

因此，非零段数  $f(p)$  = 满足上述条件的位置数。

**差分数组优化**：

对每个位置  $i$ ，分析它在哪些阈值  $p$  下充当左端点：

- $i=1$ （首位）：当  $1 \leq p \leq a_1$  时，位置 1 是左端点。  
→ 在差分数组 `diff` 上：`diff[1]++`，`diff[a[1]+1]--`
- $i > 1$  且  $a_i > a_{i-1}$ ：当  $a_{i-1}+1 \leq p \leq a_i$  时，位置  $i$  是左端点。  
→ `diff[a[i-1]+1]++`，`diff[a[i]+1]--`
- $i > 1$  且  $a_i \leq a_{i-1}$ ：位置  $i$  永远不会是左端点（ $a_{i-1} \geq a_i$  意味着若  $a_i \geq p$  则  $a_{i-1} \geq p$ ，故  $i$  不是左端点）。

最后对 `diff` 求前缀和，得到  $f(p)$ ，找最大值对应的最小  $p$ 。

```

```

### ### 完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n;
cin >> n;
vector<int> a(n);
for (auto& v : a) cin >> v;

// diff[p] 表示阈值 p 时非零段数量的变化量
vector<int> diff(n + 2, 0);

// 首位元素：是左端点当 1 <= p <= a[0]
diff[1]++;
if (a[0] + 1 <= n + 1) diff[a[0] + 1]--;

// 其余元素
for (int i = 1; i < n; i++) {
    if (a[i] > a[i - 1]) {
        // 位置 i 是左端点当 a[i-1]+1 <= p <= a[i]
        diff[a[i - 1] + 1]++;
        if (a[i] + 1 <= n + 1) diff[a[i] + 1]--;
    }
}

// 前缀和求 f(p), 找最小的最优 p
int maxCount = 0, bestP = 1;
int cur = 0;
for (int p = 1; p <= n; p++) {
    cur += diff[p];
    if (cur > maxCount) {
        maxCount = cur;
        bestP = p;
    }
}

cout << bestP << '\n';
return 0;
}
'''

```

****复杂度****： $O(n)$ 时间， $O(n)$ 空间。

202103-1：序列查询

****届次****：2021年3月（第25次 CSP-J/S）第一题

****满分****：100 分

****算法类型****： ****数学 / 模拟****

****算法手册****：→ [§ 1 暴力枚举 / 模拟](ALGORITHMS.md#1-暴力枚举--模拟)

题目描述

给定正整数 N 和严格递增序列 $A = \{a_1, a_2, \dots, a_n\}$ ($0 < a_1 < a_2 < \dots < a_n < N$)。

定义函数（约定 $a_0 = 0$ ）：

$$f(x) = \max\{i \mid 0 \leq i \leq n, a_i \leq x\}$$

即 $f(x)$ 是序列中不超过 x 的最大下标（0-indexed 从 $a_0 = 0$ 开始算）。

计算：

$$\text{ans} = \sum_{x=0}^{N-1} f(x)$$

输入格式

第一行：两个正整数 N ($1 \leq N \leq 10^9$) 和 n ($0 \leq n \leq 10^5$)

第二行： n 个正整数 $a_1, a_2, \dots, a_n$ （严格递增，均 $< N$ ）

若 $n = 0$ ，则第二行可以省略。

输出格式

一行，输出 ans 。

样例

****输入****

'''

7 3

1 3 5

'''

****输出****

'''

12

'''

****手工验证**：**

x	0	1	2	3	4	5	6
	----	---	---	---	---	---	---
$f(x)$	0	1	1	2	2	3	3

$$\text{sum} = 0 + 1 + 1 + 2 + 2 + 3 + 3 = 12 \quad \checkmark$$

解题思路

****按区间分段**：**对于 $x \in [a_i, a_{i+1}-1]$ ， $f(x)$ 恒等于 i （其中 $a_0=0$ ， $a_{n+1}=N$ ）。

$$f(x) = \sum_{i=0}^n i \cdot (a_{i+1} - a_i)$$

这样将 $O(N)$ 枚举优化为 $O(n)$ 计算，对 $N \leq 10^9$ 的情形至关重要。

注意 $a_{n+1} - a_n$ 可能很大，结果需要 `long long`。

完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 long long N;
 int n;
 cin >> N >> n;

 vector<long long> a(n + 2);
 a[0] = 0;
 for (int i = 1; i <= n; i++) cin >> a[i];
 a[n + 1] = N; // 哨兵

 long long ans = 0;
 for (int i = 0; i <= n; i++) {
 ans += (long long)i * (a[i + 1] - a[i]);
 }

 cout << ans << '\n';
 return 0;
}
```
```

****复杂度**：** $O(n)$ 时间， $O(n)$ 空间。

202012-2：期末预测之最佳阈值

****届次**：**2020年12月（第24次 CSP-J/S）第二题

****满分**：**100 分

****算法类型**：** ****排序 / 前缀和 / 枚举****

****算法手册**：**→ [§ 8 前缀和 / 差分 / 线段树](ALGORITHMS.md#8-前缀和--差分--线段树)

题目描述

共有 n 名学生，每名学生有一个**预测分** p_i 和**实际结果** r_i (0 表示未通过, 1 表示通过)。

选取正整数阈值 T ：

- 若 $p_i \geq T$ ，预测该学生通过
- 若 $p_i < T$ ，预测该学生不通过

定义**预测正确数**为预测结果与实际结果一致的学生数。

求使预测正确数最多的最小阈值 T (T 为正整数)。

输入格式

第一行：正整数 n ($1 \leq n \leq 1000$)

接下来 n 行，每行两个整数 p_i ($0 \leq p_i \leq 150$) 和 r_i (0 或 1)

输出格式

一行，输出最小最优阈值 T 。

样例

****输入****

'''

4
90 1
60 0
45 1
30 0

'''

****输出****

'''

31

'''

****说明**：**

- $T=1$ ：全部预测通过，正确 2 人 (90✓、45✓)，总正确 = 2
- $T=31$ ： $p \geq 31$ 的有 $\{90, 60, 45\}$ ，预测通过 $\rightarrow 90(r=1\checkmark)$ 、45($r=1\checkmark$) 正确； $p < 31$ 的有 $\{30\}$ ，预测不通过 $\rightarrow 30(r=0\checkmark)$ 正确；总正确 = **3**
- $T=61$ ： $p \geq 61$ 的有 $\{90\}$ ，预测通过 $\rightarrow 90(r=1\checkmark)$ ； $p < 61$ 的有 $\{60, 45, 30\}$ ，预测不通过 $\rightarrow 60(r=0\checkmark)$ 、30($r=0\checkmark$) 正确；总正确 = 3
- $T=31$ 和 $T=61$ 均正确 3 人，但 $T=31 < T=61$ ，故输出 **31**

解题思路

****暴力枚举**** ($O(n^2)$, $n \leq 1000$ 完全够用) : 枚举所有可能的阈值 T , 对每个 T 统计正确数。

****优化枚举**** ($O(n \log n)$) : 关键洞察——不同阈值 T 实际上只有 $n+1$ 种不同的分法 (按排序后的分割点)。

排序 + 前缀和 :

1. 将学生按 p_i 升序排序
2. 设 $\text{prefix_pass1}[k]$ = 前 k 名学生中 $r_i=1$ 的人数
3. 对于 "前 k 名在低组 (预测不通过), 后 $n-k$ 名在高组 (预测通过)" 的分法 :
 - 阈值 = $p[k-1] + 1$ (若 $k > 0$), 或 $T = 1$ (若 $k = 0$)
 - 预测正确数 = (高组中 $r=1$) + (低组中 $r=0$)
= $(\text{total_pass1} - \text{prefix_pass1}[k]) + (k - \text{prefix_pass1}[k])$
4. 按阈值从小到大枚举 (最先达到最大正确数的即为最小 T)

****注意**** : 若多个学生 p_i 相同, 必须同在一组 (不可拆分), 只能在同分组的边界处切割。

完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 cin >> n;
 vector<pair<int, int>> students(n); // (p[i], r[i])
 for (auto& [p, r] : students) cin >> p >> r;

 // 按预测分升序排序
 sort(students.begin(), students.end());

 int total_pass1 = 0;
 for (auto& [p, r] : students) total_pass1 += r;

 int bestCorrect = 0, bestT = 1;

 // 枚举分割点 k : 前 k 名在低组 (预测不通过), 后 n-k 名在高组 (预测通过)
 // T=1 时 k=0 : 所有人在高组
 int pre_pass1 = 0; // 低组中 r=1 的人数

 // k=0 : 阈值 T=1, 所有人预测通过
```

```

{
 int correct = total_pass1; // 高组全部是 pass1
 bestCorrect = correct;
 bestT = 1;
}

int i = 0;
while (i < n) {
 // 将所有 p[i] 相同的学生一起加入低组（不可拆分）
 int cur_score = students[i].first;
 int j = i;
 while (j < n && students[j].first == cur_score) {
 pre_pass1 += students[j].second;
 j++;
 }
 // 低组现在有 j 名学生，阈值 T = cur_score + 1
 int T = cur_score + 1;
 int pass1_high = total_pass1 - pre_pass1;
 int pass0_low = j - pre_pass1;
 int correct = pass1_high + pass0_low;

 // 仅在严格更优时更新（保证最小 T）
 if (correct > bestCorrect) {
 bestCorrect = correct;
 bestT = T;
 }
 i = j;
}

cout << bestT << '\n';
return 0;
}
'''

```

**\*\*复杂度\*\***： $O(n \log n)$  时间（排序）， $O(n)$  空间。

---

## 202309-1：坐标变换(I)

**\*\*届次\*\***：2023年9月（第31次 CSP-J/S）第一题

**\*\*满分\*\***：100 分

**\*\*算法类型\*\***：      **\*\*数学 / 模拟（旋转 + 缩放）\*\***

**\*\*算法手册\*\***：→ [ § 1 暴力枚举 / 模拟](ALGORITHMS.md#1-暴力枚举--模拟)

---

### 题目描述

给定  $n$  次坐标变换操作，以及  $m$  个查询点，对每个查询点依次应用全部  $n$  次操作，输出最终坐标。

操作分两种类型：

- \*\*类型 1\*\*：逆时针旋转  $\theta$  度（以原点为中心）
- \*\*类型 2\*\*：坐标按比例  $r$  缩放（即  $(x, y) \rightarrow (rx, ry)$ ）

### 输入格式

第一行：两个正整数  $n, m$  ( $1 \leq n, m \leq 1000$ )

接下来  $n$  行，每行描述一个操作：`1  $\theta$ ` ( $0 \leq \theta \leq 360$ , 整数) 或 `2  $r$ ` ( $1 \leq r \leq 100$ , 整数)

接下来  $m$  行，每行两个实数  $x_0, y_0$  ( $|x_0|, |y_0| \leq 100$ )

### 输出格式

$m$  行，每行输出一个查询点经过所有操作后的坐标  $(x, y)$ ，保留 2 位小数。

### 样例

**\*\*输入\*\***

```
3 2
1 90
2 2
1 45
0 1
1 0
'''
```

**\*\*输出\*\***

```
'''
-2.83 2.83
2.00 0.00
'''
```

**\*\*手工验证（点 (0,1)）\*\*：**

1. 旋转  $90^\circ$ ： $(0,1) \rightarrow (-1,0)$
2. 缩放 2： $(-1,0) \rightarrow (-2,0)$
3. 旋转  $45^\circ$ ： $(-2,0) \rightarrow (-\sqrt{2}, -\sqrt{2}) \approx (-1.41, -1.41)$

> 注：实际输出结果取决于精确计算，此处为示意。

---

### 解题思路

直接模拟两种变换：

**\*\*旋转  $\theta$  度\*\***（逆时针）：

$$x' = x \cos \theta - y \sin \theta, \quad y' = x \sin \theta + y \cos \theta$$

**\*\*缩放 \$r\$\*\* :**

$$x' = rx, \quad y' = ry$$

因为  $n, m \leq 1000$ ，直接对每个查询点应用全部操作，总复杂度  $O(n \times m) = O(10^6)$ ，完全够用。

**\*\*注意\*\***：输入角度是度数，C++ 三角函数接受弧度，需要转换： $\text{rad} = \theta \times \pi / 180$ 。

---

### 完整代码

```

`cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 const double PI = acos(-1.0);

 int n, m;
 cin >> n >> m;

 vector<int> type(n);
 vector<double> param(n);
 for (int i = 0; i < n; i++) {
 cin >> type[i] >> param[i];
 }

 cout << fixed << setprecision(2);
 for (int q = 0; q < m; q++) {
 double x, y;
 cin >> x >> y;
 for (int i = 0; i < n; i++) {
 if (type[i] == 1) {
 // 逆时针旋转 param[i] 度
 double theta = param[i] * PI / 180.0;
 double nx = x * cos(theta) - y * sin(theta);
 double ny = x * sin(theta) + y * cos(theta);
 x = nx; y = ny;
 } else {
 // 坐标按比例缩放
 x *= param[i];
 y *= param[i];
 }
 }
 cout << x << ' ' << y << '\n';
 }
}

```

```
}
return 0;
}
'''
```

**\*\*复杂度\*\***： $O(n \times m)$  时间， $O(n)$  空间。

---

## 202303-2：垦田计划

**\*\*届次\*\***：2023年3月（第30次 CSP-J/S）第二题

**\*\*满分\*\***：100 分

**\*\*算法类型\*\***：        **\*\*二分答案\*\***

**\*\*算法手册\*\***：→ [ § 2 二分查找 / 二分答案](ALGORITHMS.md#2-二分查找--二分答案)

---

### 题目描述

共有  $n$  块待开垦的田地。第  $i$  块田地自然开垦需要  $d_i$  天，但可以投入额外人力缩短工期：每缩短 1 天需要额外花费  $c_i$  单位预算；第  $i$  块田地工期最短只能缩短到  $k_i$  天（不能再短）。

所有田地**\*\*同时\*\***开始开垦，总开发时间 = 所有田地完成时的最大值（即  $\max_i \text{实际天数}_i$ ）。

现有预算上限  $W$ ，求总开发时间的最小值。

### 输入格式

第一行：两个正整数  $n$ （ $1 \leq n \leq 10^5$ ）和  $W$ （ $0 \leq W \leq 10^9$ ）

接下来  $n$  行，第  $i$  行三个整数  $d_i$ 、 $k_i$ 、 $c_i$ （ $1 \leq k_i \leq d_i \leq 10^5$ ， $1 \leq c_i \leq 10^4$ ）

### 输出格式

一行，输出最小总开发时间。

### 样例

**\*\*输入\*\***

'''

4 10

8 2 2

5 3 4

6 4 3

4 1 5

'''

**\*\*输出\*\***

'''

5

```

****验证（目标天数 $T=5$ ）**：**

| 田地 | d_i | k_i | c_i | 实际天数 | 额外花费 |
|----|-------|-------|-------|------|----------------------|
| 1 | 8 | 2 | 2 | 5 | $(8-5) \times 2 = 6$ |
| 2 | 5 | 3 | 4 | 5 | 0（已经 ≤ 5 ） |
| 3 | 6 | 4 | 3 | 5 | $(6-5) \times 3 = 3$ |
| 4 | 4 | 1 | 5 | 4 | 0（已经 ≤ 5 ） |

总花费 $= 6 + 0 + 3 + 0 = 9 \leq 10$ ✓ 可行。

$T=4$ 时：田地1花费 $(8-4) \times 2 = 8$ ，田地2 $(5-4) \times 4 = 4$ ，田地3 $(6-4) \times 3 = 6$ ，总 $18 > 10$ 不可行。

故答案为 5 。

解题思路

****二分答案**：**答案（总开发时间 T ）具有单调性—— T 越大，所需预算越少，越容易满足条件； T 越小，所需预算越多。

因此二分 T ：

- ****下界 lo **：** $\max(k_i)$ （任何字段都不能比其最短时间更短）
- ****上界 hi **：** $\max(d_i)$ （不投入任何预算的自然时间）

****`check(T)`**：**给定目标天数 T ，计算使所有田地完工时间 $\leq T$ 所需的最小额外预算。

$$\text{cost}(T) = \sum_{i=1}^n \max(0, (d_i - T) \times c_i)$$

- 若 $d_i \leq T$ ：该田地不需要额外投入，花费 0
- 若 $k_i \leq T < d_i$ ：将该田地缩短到 T 天，花费 $(d_i - T) \times c_i$
- 若 $T < k_i$ ：因为 $lo = \max(k_i)$ ，这种情况在二分范围内不会出现

若 $\text{cost}(T) \leq W$ ，则 T 可行，尝试更小的值（往左二分）；否则不可行，往右扩大 T 。

完整代码

```
```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

int n;
long long W;
cin >> n >> W;

vector<long long> d(n), k(n), c(n);
for (int i = 0; i < n; i++) cin >> d[i] >> k[i] >> c[i];

// 二分范围
long long lo = *max_element(k.begin(), k.end()); // T 不能低于任何 k[i]
long long hi = *max_element(d.begin(), d.end()); // 最多不缩短任何田地

// check(T) : 将所有田地完工时间压缩到 T 天以内，所需总预算
auto check = [&](long long T) -> bool {
 long long cost = 0;
 for (int i = 0; i < n; i++) {
 if (d[i] > T) {
 cost += (d[i] - T) * c[i];
 if (cost > W) return false; // 提前剪枝，避免溢出
 }
 }
 return cost <= W;
};

long long ans = hi;
while (lo <= hi) {
 long long mid = lo + (hi - lo) / 2;
 if (check(mid)) {
 ans = mid; // mid 可行，尝试更小
 hi = mid - 1;
 } else {
 lo = mid + 1; // mid 不可行，需要更多天数
 }
}

cout << ans << '\n';
return 0;
}

```

**\*\*复杂度\*\***： $O(n \log (\max d_i))$  时间， $O(n)$  空间。

---

## ## 各题算法类型对照表

算法类型	对应真题	手册章节
模拟 / 混合进制	202209-1 如此编码	[ § 1](ALGORITHMS.md#1-暴力枚举--模拟)

数学 / 模拟	202203-1 归一化处理	[ § 1](ALGORITHMS.md#1-暴力枚举--模拟)
数学 / 模拟	202103-1 序列查询	[ § 1](ALGORITHMS.md#1-暴力枚举--模拟)
数学 / 旋转缩放	\*\*202309-1 坐标变换(I)\*\*	[ § 1](ALGORITHMS.md#1-暴力枚举--模拟)
贪心	202109-1 数组推导	[ § 3](ALGORITHMS.md#3-贪心算法)
二分答案	\*\*202303-2 垦田计划\*\*	[ § 2](ALGORITHMS.md#2-二分查找--二分答案)
拓扑排序	202203-2 训练计划	[ § 7](ALGORITHMS.md#7-图论基础)
差分数组	202109-2 非零段划分	[ § 8](ALGORITHMS.md#8-前缀和--差分)
排序+前缀和	202012-2 期末预测之最佳阈值	[ § 8](ALGORITHMS.md#8-前缀和--差分)

---

## ## 刷题建议

**\*\*第一阶段（目标 200 分）\*\***：先做所有 T1 题（确保每道不超时拿满分）

1. 202203-1 归一化处理（最简单，纯公式模拟）
2. 202109-1 数组推导（贪心思维，读懂推导过程）
3. \*\*202309-1 坐标变换(I)\*\*（三角函数模拟，注意度→弧度转换）

**\*\*第二阶段（目标 300 分）\*\***：攻克 T2 题（掌握一两种 T2 算法即可）

4. 202012-2 期末预测之最佳阈值（排序+枚举，逻辑清晰）
5. 202109-2 非零段划分（差分数组技巧）
6. \*\*202303-2 垦田计划\*\*（二分答案入门题，思路清晰）
7. 202203-2 训练计划（拓扑排序入门）

> **\*\*2024 – 2025 年新题\*\***：请前往 [CCF 官方网站](https://www.ccf.org.cn/csp/) 或 [洛谷 CSP 专题](https://www.luogu.com.cn/problem/list?tag=csp-j,csp-s) 获取最新题目。近年 T1 仍以模拟为主，T2 以贪心/排序/二分/简单 DP 为主，与本文档中的算法类型高度重叠。

---

\*配合[算法速查手册](ALGORITHMS.md)和[300+ 分方案](README.md)使用效果最佳。\*